

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>

using namespace std;

/*
Required implementation
Report a small timing table and explain the observed best, average, and worst behavior.
*/
vector<int> Bubble(const vector<int>& values){
    bool swapped;
    vector<int> sorted_values = values;

    for(int i = 0; i < sorted_values.size() - 1; ++i){
        for(int j = 0; j < sorted_values.size() - i - 1; ++j){
            if(sorted_values[j] > sorted_values[j+1]){
                swap(sorted_values[j], sorted_values[j+1]);
                swapped = true;
            }
        }
        if(!swapped){
            break;
        }
    }
    return sorted_values;
    //for loop around array with a decreasing vector length each loop

    //check index i with index i+1

    //if index i is bigger than index j, swap

    //else continue

    //i++
}

vector<int> Selection(const vector<int>& values){
    //Seclection sort goes through the vector and chooses a selected element and goes through the
    vector finding the smallest one to "swap"
    //if going through the array there is a smaller number, it becomes the new "swap" until we reach
    the end of the array

    //We will need a double for loop, one for the selected element we are in, and another one to
    search through the array finding the "swap" element
    vector<int> sorted_values = values;
    for(int i = 0; i < sorted_values.size() - 1; ++i){
        int min_Index = i;

        for(int j = i + 1 ; j < sorted_values.size() ; ++j){
            if(sorted_values[min_Index] > sorted_values[j]){
                min_Index = j;
            }
        }
        swap(sorted_values[i], sorted_values[min_Index]);
    }
}

```

```

    }
    return sorted_values;
}

vector<int> Insertion(const vector<int>& values){
    vector<int> sorted_values = values;
    //Insertion will start on the left side and compare the element next to it, if its in order,
    continue. If not we swap the elements, and
    //repeat the same action if the anterior elements are greater than the current chosen element.

    //We will use two loops, the outside loop is in control of choosing the insertion element.
    //Our i incicies will be ahead by 1 wheras j is always behind i
    //Second loop is a while loop controlling the swapping and deincramentation while j is >= 0, and
    the j element is greater than our insert "i"
    // If the current insertion element is less than the linear search element
    //swap both elements
    //deincrament j by one
    //outside while loop we insure the last element of the vector is added to the sorted vector

    for(int i = 1; i < sorted_values.size(); ++i){
        int insert = sorted_values[i];
        int j = i-1;

        while(j >= 0 && sorted_values[j] > insert){
            sorted_values[j+1] = sorted_values[j];
            j--;
        }
        sorted_values[j+1] = insert;
    }
    return sorted_values;
}

vector<int> quickSort(const vector<int>& values, int low, int high){
    /*
    ***Round down pivot index
    quickSort utilizes a pivot type search where, the pivot will be the middle of the vector "most of
    the time", and we treat the pivot as the
    middle split where we start grouping. We take each half and make "pointers" for each end, we will
    then swap and compare as we get closer to the pivot

    - Take middle element and assign it as pivot value
    - Assign low and high values to i and j repectivly so that we can close in to the pivot
    incrementally

    - Make while loop that checks i and j "pointers" are closing in
      - make while loop to search for element on the left side to swap, when found, i++

      - make while loop to search for element on the right side to swap, when found, j--

    - if pointers havent crossed, swap elements
    - recursivly call the method again to sort the rest of the unsorted
    */
    vector<int> sorted_values = values;
    int i = low;
    int j = high;
    int pivot_value = sorted_values[low + (high - low) / 2];

    while( i <= j){
        while(sorted_values[i] < pivot_value){
            i++;

```

```

    }

    while(sorted_values[j] > pivot_value){
        j--;
    }

    if(i <= j){
        swap(sorted_values[i], sorted_values[j]);
        i++;
        j--;
    }
}

if(low < j){
    sorted_values = quickSort(sorted_values, low, j);
}
if(i < high){
    sorted_values = quickSort(sorted_values, i, high);
}

return sorted_values;
}

//Check if my sorted functions worked
bool isSorted(const vector<int>& values){
    bool sorted = true;

    for(int i = 0; i < values.size() - 1; ++i){
        if(values[i] > values[i + 1]){
            return false;
        }
    }
    return true;
}

int main(){
    vector<int> vectRand =
{29,10,14,37,14,48,10,456,15,10,10,16,1,89,0,10,4,10,74,9,14,9,3,0,1,78,20,13};
    vector<int> vectLG = {1, 2, 3, 3, 4, 4, 5, 5, 6, 7, 7, 8, 8, 9, 9, 10, 11, 11, 11, 12, 12, 12,
14, 143, 14, 34, 34, 34, 35, 56, 56};
    vector<int> vectGL = {99, 90, 78, 56, 56, 56, 56, 56, 33, 33, 33, 22, 23, 23, 23, 23, 11, 11, 10,
9, 9, 9, 8, 7, 7, 6, 6, 5, 4, 4, 3, 3, 2, 2, 1};

    cout << "-----Program started for Bubble sort!-----" << endl;
    cout << "#### SORTING RANDOM VECTOR ####" << endl;
    vector<int> sorted1 = Bubble(vectRand);
    bool response = isSorted(sorted1);

    for (int value : sorted1) {
        cout << value << ' ';
    }

    cout << endl;
    cout << response << endl;
    cout << "Program ended sorting!" << endl;

    cout << "#### SORTING LEAST TO GREATEST VECTOR ####" << endl;

    sorted1 = Insertion(vectLG);
    response = isSorted(sorted1);

    for (int value : sorted1) {

```

```

        cout << value << ' ';
    }

    cout << endl;
    cout << response << endl;
    cout << "Program ended sorting!" << endl;

    cout << "#### SORTING GREATEST TO LEAST VECTOR ####" << endl;

    sorted1 = Insertion(vectGL);
    response = isSorted(sorted1);

    for (int value : sorted1) {
        cout << value << ' ';
    }

    cout << endl;
    cout << response << endl;
    cout << "Program ended Bubble sort!" << endl;

    //////////////////////////////////////
    //////////////////////////////////////

    cout << "-----Program started Selection sort!-----" << endl;

    cout << "#### SORTING RANDOM VECTOR ####" << endl;
    sorted1 = Selection(vectRand);
    response = isSorted(sorted1);

    for (int value : sorted1) {
        cout << value << ' ';
    }

    cout << endl;
    cout << response << endl;
    cout << "Program ended sorting!" << endl;

    cout << "#### SORTING LEAST TO GREATEST VECTOR ####" << endl;

    sorted1 = Selection(vectLG);
    response = isSorted(sorted1);

    for (int value : sorted1) {
        cout << value << ' ';
    }

    cout << endl;
    cout << response << endl;
    cout << "Program ended sorting!" << endl;

    cout << "#### SORTING GREATEST TO LEAST VECTOR ####" << endl;

    sorted1 = Selection(vectGL);
    response = isSorted(sorted1);

    for (int value : sorted1) {
        cout << value << ' ';
    }

    cout << endl;
    cout << response << endl;
    cout << "Program ended Selection sort!" << endl;

```

```

//#####
//#####
cout << "-----Program started Insertion sort!-----" << endl;

cout << "#### SORTING RANDOM VECTOR ####" << endl;
sorted1 = Insertion(vectRand);
response = isSorted(sorted1);

for (int value : sorted1) {
    cout << value << ' ';
}

cout << endl;
cout << response << endl;
cout << "Program ended sorting!" << endl;

cout << "#### SORTING LEAST TO GREATEST VECTOR ####" << endl;

sorted1 = Insertion(vectLG);
response = isSorted(sorted1);

for (int value : sorted1) {
    cout << value << ' ';
}

cout << endl;
cout << response << endl;
cout << "Program ended sorting!" << endl;

cout << "#### SORTING GREATEST TO LEAST VECTOR ####" << endl;

sorted1 = Insertion(vectGL);
response = isSorted(sorted1);

for (int value : sorted1) {
    cout << value << ' ';
}

cout << endl;
cout << response << endl;
cout << "Program ended Insertion sort!" << endl;

//#####
//#####
int low = 0;
int high = vectRand.size() - 1;

cout << "-----Program started Quick sort!-----" << endl;

cout << "#### SORTING RANDOM VECTOR ####" << endl;
sorted1 = quickSort(vectRand, low, high);
response = isSorted(sorted1);

for (int value : sorted1) {
    cout << value << ' ';
}

cout << endl;
cout << response << endl;
cout << "Program ended sorting!" << endl;

cout << "#### SORTING LEAST TO GREATEST VECTOR ####" << endl;
int high2 = vectLG.size() - 1;

sorted1 = quickSort(vectLG, low, high2);

```

```

    response = isSorted(sorted1);

    for (int value : sorted1) {
        cout << value << ' ';
    }

    cout << endl;
    cout << response << endl;
    cout << "Program ended sorting!" << endl;

    cout << "#### SORTING GREATEST TO LEAST VECTOR ####" << endl;
    int high3 = vectGL.size() - 1;

    sorted1 = quickSort(vectGL, low, high3);
    response = isSorted(sorted1);

    for (int value : sorted1) {
        cout << value << ' ';
    }

    cout << endl;
    cout << response << endl;
    cout << "Program ended Quick sort!" << endl;

    return 0;
}

```

Time complexity Table

Sorts	Worst	Average	Best
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(n)$
Quick Sort	$O(n^2)$	$O(n \log(n))$	$O(n \log(n))$